

How I Completed Exercise 01 Using the Centos 7 VM

First, I installed and ran the Centos virtual machine image and logged in as the pre-made user. There are instructions near the top of our Moodle page for how to download this image, install it and then log in. You're not required to use the Centos VM, but it can be a convenient way to work if you don't always have access to a good internet connection.

Preparing your Environment

After logging in, the VM window should expand to give you a lot of space to work. If it doesn't, you can resize it to have a larger desktop space for the VM.

I started up a terminal window (using Applications → System Tools → Terminal from the menu at the top of the desktop). Then, in the terminal, I used the following commands to make a new directory where I wanted to work on exercise 1.

```
$ mkdir csc230
$ cd csc230
$ mkdir ex
$ cd ex
$ mkdir ex01
$ cd ex01
```

I started Firefox (using Applications → Internet → Firefox Web Browser). Inside Firefox, I visited wolfware.ncsu.edu, logged in as myself, went to the Moodle page for the course and clicked on the link for Exercise 1 Files. From there, I right-clicked on the **arithmetic.c** and **expected.txt** links to download them (using "Save Link As ..."). I saved the files to the same directory I created in the previous step. Or, if you just save them to Downloads, you can copy them from there afterward.

Instead of downloading these files from the web browser, you could also use the **scp** command given with the exercise write-up to download the needed files all at once. This would probably be faster.

Completing your Program

To edit the source file, I used emacs (an editor I often use for little programming projects). There are several editors available on the VM, including gedit, nano and vim. Emacs and vim have a lot more features than gedit, but gedit is a lot easier to get started with if you don't want to invest a lot of time learning a new editor.

```
$ emacs arithmetic.c &
```

The **&** at the end told the shell to run emacs in the background, popping up an extra window and leaving me with the terminal window to continue running commands. If you'd prefer to use one of the other editors, you can run them with commands like:

To use vi/vim	To use gedit	To use nano
---------------	--------------	-------------

```
$ vim arithmetic
```

```
$ gedit arithmetic.c &
```

```
$ nano arithmetic.c
```

In my editor, I added the missing lines of code, then saved the file. With emacs, I could have chosen “save” from the file menu to save the updated file (but, really, I pressed ctrl-c ctrl-s, because it was easier). If you’re using a different editor, the way you edit the file and save will be different.

Then, I tried compiling the program from the terminal window:

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
```

If my program hadn't compiled cleanly, I'd have gone back to the editor and fixed whatever was wrong. Since it compiled OK, I'm ready to try it out. In shell code below, I'm using the `tee` command to send output to the terminal window, while still capturing a copy of the output to a file. This is handy for seeing the output and still capturing a copy to a file so we can later check it against the expected output. After running the arithmetic program, I check its exit status to make sure it's zero, then I compare the output against the expected output file.

```
$ ./arithmetic | tee output.txt
500000000500000000
$ echo $?
0
$ diff output.txt expected.txt
```

Both of these tests look good, so now I have a choice. I can turn it in as is and hope it will work the same on a common platform machine, or I can try it out on a real common platform machine first, just to be sure.

Checking on the Common Platform

Let's take the safe path.

I used **sftp** to upload the source file and expected output to my file space in NCSU Drive. It's a command-line tool that lets you upload or download files from a remote system. If you're accustomed to using FileZilla, sftp does a similar job, but, instead of giving you a graphical interface for uploading and downloading files, it lets you copy files back and forth by typing commands at a prompt. Or, if you've set up access to your NCSU Drive space on your host computer, you could probably just copy your completed files using the file browser.

For copying the files with sftp, you should be able to use commands like the following, maybe with a few changes like replacing *unity-id* with your own unity ID or creating different directories if you'd like to organize your files differently.

```
$ sftp unity-id@remote.eos.ncsu.edu
Connecting to remote.eos.ncsu.edu...
unity-id@remote.eos.ncsu.edu's password:
sftp> mkdir csc230
sftp> cd csc230
```

```
sftp> mkdir ex
sftp> cd ex
sftp> mkdir ex01
sftp> cd ex01
sftp> put arithmetic.c
Uploading arithmetic.c to /mnt/ncsudrive/.../ex01/arithmetic.c
sftp> put expected.txt
Uploading expected.txt to /mnt/ncsudrive.../ex01/expected.txt
sftp> exit
```

Then, I used **ssh** to log in to an EOS linux machine to try out my program. If you're accustomed to using putty, or MobaXterm, ssh does the same job (in fact, these programs are just an examples of what we'd call a ssh client). In the commands below, I'm logging in using ssh, then, after authenticating with my NCSU credentials, I change directory to the place where I uploaded my exercise 1 files. I compile the program and then run it. Finally, I check the exit status and compare the program's output against the expected output.

```
$ ssh unity-id@remote.eos.ncsu.edu
unity-id@remote.eos.ncsu.edu's password:
Last login: ... from ...
eos$ cd csc230/ex/ex01
eos$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
eos$ ./arithmetic | tee output.txt
50000000005000000000
eos$ echo $?
0
eos$ diff output.txt expected.txt
eos$ exit
```

Good, it worked just as well on the EOS linux machine as it did on the VM.

Submitting your Solution

I'm ready to turn it in. After starting Firefox in the VM, I went to the CSC 230 Moodle home page, and chose the assignment for exercise 01. This took me to a submission page where I chose "Add Submission". From there, you can drag a copy of your solution into the submission box (use the Applications → Accessories → Files application to browse around the file system). Or, you can just click on the blue arrow then choose "Upload a File" on the left, then choose "Browse" to choose your file via the web browser. Either way, select your arithmetic.c file for submission then choose "Save changes" down near the bottom of the web page.